Case Study 1: Java-Based Configuration

Project Title: Online Food Ordering System

Configuration Type: Java-based Spring Configuration

POJO Classes: Restaurant and Customer

Scenario: An online food ordering platform allows customers to order food from various restaurants. The system must manage customer information and restaurant offerings. The logic for selecting restaurants and placing orders is handled in a service class. Java-based configuration is used to wire beans explicitly.

Customer.java

```java
package com.example.food;

public class Customer {
    private String name;
    private String preferredCuisine;

    public Customer(String name, String preferredCuisine) {
        this.name = name;
        this.preferredCuisine = preferredCuisine;
    }

    public String getPreferredCuisine() {
        return preferredCuisine;
    }

    public String getName() {
        return name;
    }

}
```

Restaurant.java

```java
package com.example.food;

import java.util.List;

public class Restaurant {
    private String name;
    private String location;
    private List<String> availableCuisines;

    public Restaurant(String name, String location, List<String> availableCuisines) {
        this.name = name;
        this.location = location;
        this.availableCuisines = availableCuisines;
    }

    public String getName() {
        return name;
    }

    public String getLocation() {
        return location;
    }

    public List<String> getAvailableCuisines() {
        return availableCuisines;
    }
}
```

FoodOrderService.java

```java
package com.example.food;

public class FoodOrderService {
    private Customer customer;
    private Restaurant restaurant;

    public FoodOrderService(Customer customer, Restaurant restaurant) {
        this.customer = customer;
        this.restaurant = restaurant;
    }

    public void placeOrder() {
        if (restaurant.getAvailableCuisines().contains(customer.getPreferredCuisine())){
            System.out.println("Order placed at " + restaurant.getName());
            System.out.println(customer.getPreferredCuisine()+" Cuisine by " + customer.getName());
            System.out.println();
        }
    }
}
```

MainApp.java

```java
package com.example.food;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class MainApp {
    public static void main(String[] args) {
        ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
        FoodOrderService service = context.getBean(FoodOrderService.class);
        service.placeOrder();
    }
}
```

AppConfig.java

```java
package com.example.food;

import java.util.Arrays;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AppConfig {

    @Bean
    public Customer customer() {
        return new Customer("rpk","Indian");
    }

    @Bean
    public Restaurant restaurant() {
        return new Restaurant("Biryani House", "Cyberabad", Arrays.asList("Indian", "Chinese"));
    }

    @Bean
    public FoodOrderService foodOrderService() {
        return new FoodOrderService(customer(), restaurant());
    }
}
```

<terminated> MainApp (1) [Java Applicatio

```
Order placed at Biryani House
Indian Cuisine by rpk
```

Case Study 2: Annotation-Based Configuration

Project Title: Smart Home Automation System

Configuration Type: Annotation-based Spring Configuration

POJO Classes: Device and User

Scenario: A smart home system manages various IoT devices like lights, fans, and ACs. Users can control these devices through an application. Each user can register and manage multiple devices. Spring annotations like @Component, @Autowired, and @Service are used to auto-wire dependencies and manage components

User.java

```java
package com.homeautomation;

import org.springframework.stereotype.Component;

@Component
public class User {
    private String name = "rpk";
    private String homeId = "home236";

    public String getName() {
        return name;
    }
    public String getHomeId() {
        return homeId;
    }
}
```

Device.java

```java
package com.homeautomation;

import org.springframework.stereotype.Component;

@Component
public class Device {
    private String deviceType = "Washing Machine";
    private boolean status = false;

    public String getDeviceType() {
        return deviceType;
    }
    public boolean isStatus() {
        return status;
    }

    public void turnOn() {
        status = true;
        System.out.println(deviceType + " is turned ON.");
    }
    public void turnOff() {
        status = false;
        System.out.println(deviceType + " is turned OFF.");
    }
}
```

HomeAutomationService.java

```java
package com.homeautomation;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class HomeAutomationService {
    @Autowired
    private User user;
    @Autowired
    private Device device;

    public void controlDevice() {
        System.out.println("User: "+user.getName() +" controling the device");
        device.turnOff();
        device.turnOn();
    }
}
```

AppConfig.java

```java
package com.homeautomation;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages = "com.homeautomation")
public class AppConfig {

}
```

MainApp.java

```java
package com.homeautomation;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class MainApp {
    public static void main(String[] args) {

        ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
        HomeAutomationService service = context.getBean(HomeAutomationService.class);
        service.controlDevice();
    }

}
```

Problems | Console

```
<terminated> MainApp (2) [Java Applicatio
User: rpk controling the device
Washing Machine is turned OFF.
Washing Machine is turned ON.
```